Data Preparation

GitHub Repository: https://github.com/christophrrrrr/ML-fundamentals-2026

This notebook executes data preparation and feature engineering tasks for the UCI Bank Marketing Dataset (`bank-additional.csv`), adhering to data leakage prevention principles.

> **Individual Assignment I: Machine Learning Foundation**

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score, recall_score,
    ConfusionMatrixDisplay, classification_report, f1_score,
    precision_recall_curve, average_precision_score
)
from sklearn.feature_selection import VarianceThreshold
import warnings
warnings.filterwarnings('ignore')
from imblearn.over_sampling import RandomOverSampler
```

1. Identifying the Prediction Target

Lecture material: Lecture 1 (Problem Formulation), Lecture 2 (Data Inspection).

Target Selection: The target variable is `y`. `y` records "yes" or "no" indicating whether the client subscribed to a term deposit. This aligns with the stated objective of the direct marketing campaigns.

Invalid Alternatives: Three other variables might superficially appear to be valid prediction targets but must not be used:

1. **`duration`**: Only known after the call ends — unavailable at prediction time. More critically, using it as a feature would introduce target leakage since call duration is strongly determined by whether the client ultimately subscribes. Dropped entirely before any further processing.
2. **`poutcome`**: Records the result of the *previous* marketing campaign. It describes an event that has already occurred and is available as an input at

prediction time — it is an input feature, not the target.

3. **`campaign`**: Records the number of contacts during the current campaign. This is a campaign execution variable, not a business outcome. The goal is predicting client behaviour (`y`), not the number of calls made.

2. Data Loading and Exploration

Lecture material: Lecture 1 (Problem Formulation), Lecture 2 (Data Inspection and EDA).

`bank-additional.csv` is the 10% sample (4119 rows) randomly selected from `bank-additional-full.csv` (41188 rows). We prefer the full dataset but fall back to the 10% sample to keep computation light or if the full set is unavailable. Note that UCI bank datasets commonly use the semicolon `;` separator.

In [2]:
```
import os
full_filepath = 'data/bank-additional-full.csv'
sample_filepath = 'data/bank-additional.csv'
github_sample_url = 'https://raw.githubusercontent.com/christophrrrrr/ML-fur

if os.path.exists(full_filepath):
    print(f'Loading full dataset from: {full_filepath}')
    df = pd.read_csv(full_filepath, sep=';')
elif os.path.exists(sample_filepath):
    print(f'Loading 10% sample dataset from: {sample_filepath}')
    df = pd.read_csv(sample_filepath, sep=';')
else:
    print(f'Local instance not found. Downloading 10% sample directly from G
    df = pd.read_csv(github_sample_url, sep=';')

print(f'Number of observations: {df.shape[0]}')
print(f'Number of features: {df.shape[1]}')
print('\n--- Data Types ---')
print(df.dtypes)
print('\n--- Summary Statistics ---')
display(df.describe())
```

```
Local instance not found. Downloading 10% sample directly from GitHub reposi
tory...
Number of observations: 4119
Number of features: 21

--- Data Types ---
age                 int64
job                   str
marital               str
education             str
default               str
housing               str
loan                  str
contact               str
month                 str
day_of_week           str
duration            int64
campaign            int64
pdays               int64
previous            int64
poutcome              str
emp.var.rate      float64
cons.price.idx    float64
cons.conf.idx     float64
euribor3m         float64
nr.employed       float64
y                     str
dtype: object

--- Summary Statistics ---
```

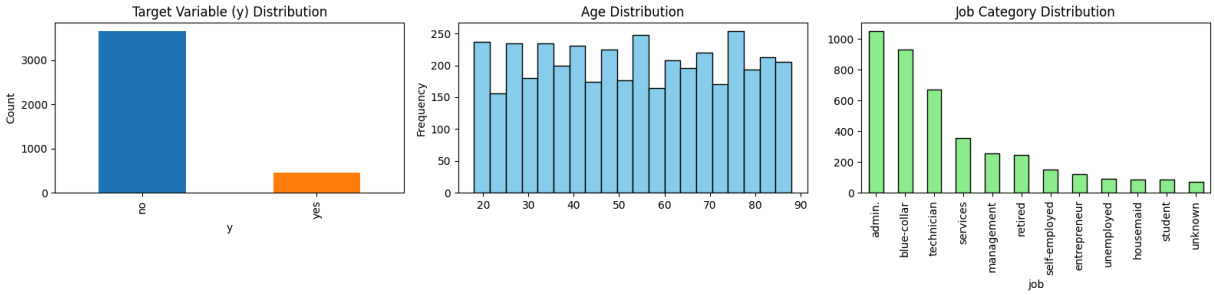
Variable Identification (raw dataset, prior to feature engineering):

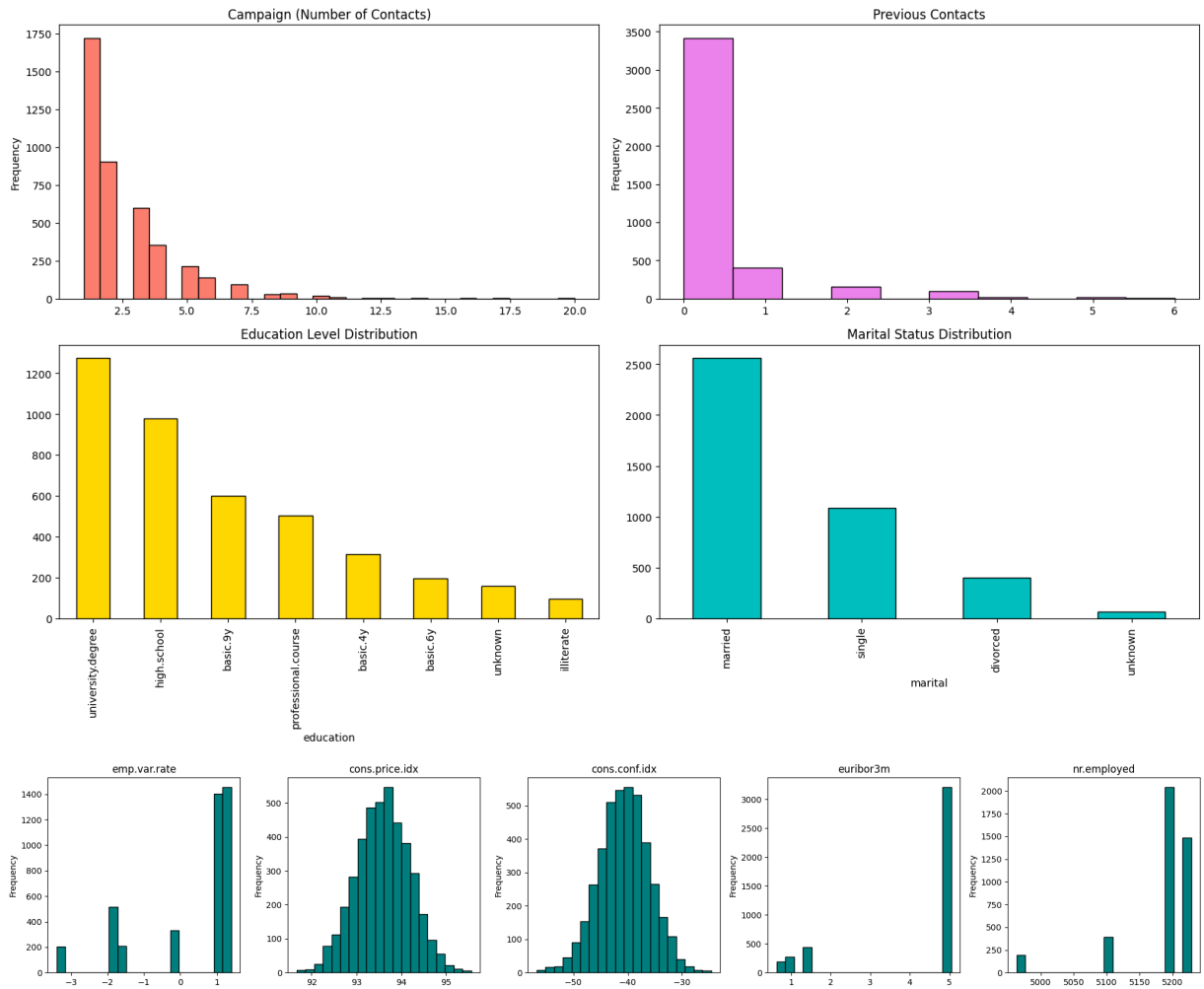
- **Numerical:** `age`, `duration`, `campaign`, `pdays`, `previous`, `emp.var.rate`, `cons.price.idx`, `cons.conf.idx`, `euribor3m`, `nr.employed`
- **Categorical:** `job`, `marital`, `education`, `default`, `housing`, `loan`, `contact`, `month`, `day_of_week`, `poutcome`, `y`

Note: `duration` is identified here for completeness but is dropped immediately due to target leakage. `pdays` is decomposed into a binary feature (`prev_contacted`) capturing whether the client was previously contacted — the raw magnitude is discarded because 96% of values are the sentinel 999.

In [3]:
```
y_counts = df['y'].value_counts()
y_pct = df['y'].value_counts(normalize=True)

# --- Figure 1: Target, Age, Job ---
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
df['y'].value_counts().plot(kind='bar', color=['#1f77b4', '#ff7f0e'], ax=axe
axes[0].set_title('Target Variable (y) Distribution')
axes[0].set_ylabel('Count')
df['age'].plot(kind='hist', bins=20, color='skyblue', edgecolor='black', ax=
```

```
axes[1].set_title('Age Distribution')
df['job'].value_counts().plot(kind='bar', color='lightgreen', edgecolor='bla
axes[2].set_title('Job Category Distribution')
plt.tight_layout()
plt.show()

# --- Figure 2: Campaign, Previous, Education, Marital ---
fig2, axes2 = plt.subplots(2, 2, figsize=(16, 10))
df['campaign'].plot(kind='hist', bins=30, color='salmon', edgecolor='black',
axes2[0, 0].set_title('Campaign (Number of Contacts)')
df['previous'].plot(kind='hist', bins=10, color='violet', edgecolor='black',
axes2[0, 1].set_title('Previous Contacts')
df['education'].value_counts().plot(kind='bar', color='gold', edgecolor='bla
axes2[1, 0].set_title('Education Level Distribution')
df['marital'].value_counts().plot(kind='bar', color='c', edgecolor='black',
axes2[1, 1].set_title('Marital Status Distribution')
plt.tight_layout()
plt.show()

# --- Figure 3: Macroeconomic Variables ---
macro_vars = ['emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m'
fig3, axes3 = plt.subplots(1, 5, figsize=(20, 4))
for i, var in enumerate(macro_vars):
    df[var].plot(kind='hist', bins=20, color='teal', edgecolor='black', ax=a
    axes3[i].set_title(var)
plt.tight_layout()
plt.show()

print('Target variable counts:')
print(y_counts)
print('\nTarget variable percentages:')
print(y_pct)
```






```
Target variable counts:
y
no     3660
yes     459
Name: count, dtype: int64

Target variable percentages:
y
no     0.888565
yes    0.111435
Name: proportion, dtype: float64
```

General Observations:

- **Class Imbalance:** Only ~10.9% of clients subscribed (`yes`). Class imbalance handling is required to prevent the model from trivializing predictions.
- **Skewed Variables:** `campaign` is right-skewed (most clients are contacted 1–3 times, with a long tail reaching 35). `previous` is zero for the majority of clients.
- **Macroeconomic Variables:** `euribor3m` and `nr.employed` show bimodal distributions, clustering into two distinct regimes likely corresponding to pre- and post-2008 economic periods. `emp.var.rate` is similarly clustered,

reinforcing that the macroeconomic block may carry redundant information — addressed formally in Feature Selection.

- **`cons.price.idx` note:** Like `euribor3m`, this is a macroeconomic aggregate known only at the population level at the time of each call. It is correlated with `euribor3m` (r≈0.65) but not above the 0.85 collinearity threshold, so it is retained. Whether it constitutes a weak leakage concern is debatable — it is collected at campaign time and does not depend on individual call outcomes, so it is treated as a valid input feature here.

3. Task Ordering

Lecture material: Lecture 2 (Data Splitting and Leakage), Lecture 5 (Preprocessing), Lecture 9 (ML Pipeline).

To prevent data leakage, data preparation tasks are executed in the following sequence:

1. **Identifying Target & Data Loading** — Raw dataset viewing only. `duration` dropped manually before any modeling.
2. **Missing Values (Structural Cleaning)** — Replace sentinel strings (`'unknown'`, `999`) with `NaN` or indicator flags. No statistics computed — safe before splitting.
3. **Data Splitting** — Stratified split on raw features. Ensures all downstream `.fit()` calls see only training data.
4. **Missing Values (Statistical Imputation)** — Medians/modes calculated from `X_train` only.
5. **Encoding Categorical Variables** — Category lists derived from `X_train` only. `handle_unknown='ignore'` for unseen categories.
6. **Feature Scaling** — Mean and variance computed over `X_train` via `.fit()`. Applied to val/test via `.transform()` only.
7. **Feature Selection** — Variance thresholds and correlation matrices computed over `X_train` only.
8. **Addressing Class Imbalance** — `RandomOverSampler` applied to training set only, after preprocessing.

Incorrect Ordering Example 1 — Scaling before Splitting: If Feature Scaling is performed before Data Splitting, the mean and standard deviation are calculated across the entire dataset, including test observations. The standardized test values inherently contain information about the central tendency of the training set. This is data leakage.

Incorrect Ordering Example 2 — Resampling before Splitting: If `RandomOverSampler` were applied to the full dataset before the split, synthetic minority samples would be generated using the entire data distribution. When

the dataset is subsequently split, some synthetic samples — constructed using observations that end up in the validation or test sets — would appear in the training set. The model trains on data derived from the test set, inflating evaluation metrics.

4. Managing Missing Values (Part 1: Identification & Sentinel Cleaning)

Lecture material: Lecture 2 (Data Inspection), Lecture 5 (Preprocessing and Pipeline Discipline).

Identification:

- Explicit missing values (`NaN`) are largely absent in this CSV.
- Implicit missing values are abundant: `'unknown'` maps strictly to missing information in categorical columns.
- In numerical columns, `pdays=999` acts as a sentinel for 'client was not previously contacted'.

Note on Leakage: Replacing `'unknown'` → `NaN` and extracting `pdays != 999` are purely structural operations — no statistics are computed. This is safe to execute before Data Splitting.

Note on `default` (19.5% missing): The high missingness rate in `default` warrants scrutiny. The pattern is unlikely to be random: banks may systematically withhold default status for certain client profiles (e.g., clients with complex credit histories). Dropping the column was considered but rejected — the 80.5% of non-missing values carry signal, and encoding missingness as a separate `'missing'` category allows the model to learn whether the *absence* of default information is itself predictive of subscription behaviour. This is a principled imputation choice, not a mechanical one.

In [4]:
```
if 'duration' in df.columns:
    df = df.drop(columns=['duration'])

missing_counts = []
for col in df.drop(columns=['y']).select_dtypes(include=['object']).columns:
    unknown_count = (df[col] == 'unknown').sum()
    if unknown_count > 0:
        missing_counts.append({
            'Variable': col,
            'Implicit Missing': unknown_count,
            '% of Total': f'{(unknown_count / len(df)) * 100:.2f}%'
        })
pdays_count = (df['pdays'] == 999).sum()
if pdays_count > 0:
    missing_counts.append({
```

```
        'Variable': 'pdays',
        'Implicit Missing': pdays_count,
        '% of Total': f'{(pdays_count / len(df)) * 100:.2f}%'
    })
missing_df = pd.DataFrame(missing_counts)
print('--- Implicit Missing Values Summary Before Structural Cleaning ---')
display(missing_df)
```

```
--- Implicit Missing Values Summary Before Structural Cleaning ---
    Variable  Implicit Missing % of Total
0        job                72      1.75%
1    marital                66      1.60%
2  education               158      3.84%
3    default               844     20.49%
4    housing               151      3.67%
5       loan               121      2.94%
6      pdays              3988     96.82%
```

In [5]:

```
# 1. Handle Categorical 'unknown'
cat_cols_raw = df.select_dtypes(include=['object']).columns
for col in cat_cols_raw:
    df[col] = df[col].replace('unknown', np.nan)

# 2. Handle 'pdays' Sentinel
# 96% of pdays values are 999 (sentinel for 'never contacted').
# Only the binary signal is extracted. The raw magnitude is NOT retained:
# with 96% missingness, median imputation would produce a near-constant
# column carrying no meaningful signal beyond prev_contacted.
df['prev_contacted'] = (df['pdays'] != 999).astype(int)
df = df.drop(columns=['pdays'])

missing_summary = df.isna().sum()
print('Missing (NaN) counts after structured cleaning:')
print(missing_summary[missing_summary > 0])
print(f"\npdays replaced by binary feature 'prev_contacted'")
print(f"prev_contacted=1: {(df['prev_contacted']==1).sum()} rows ({(df['prev
print(f"prev_contacted=0: {(df['prev_contacted']==0).sum()} rows ({(df['prev
```

```
Missing (NaN) counts after structured cleaning:
job           72
marital       66
education    158
default      844
housing      151
loan         121
dtype: int64

pdays replaced by binary feature 'prev_contacted'
prev_contacted=1: 131 rows (3.2%)
prev_contacted=0: 3988 rows (96.8%)
```

5. Data Splitting

Lecture material: Lecture 2 (Data Splitting and Leakage), Lecture 9 (ML Pipeline).

The independent features `X` and the target `y` are separated using a stratified split.

Proportions: Training: 70% | Validation: 15% | Test: 15%

Stratification: `stratify=y` is used because the target is imbalanced (~11% positives). A random split could yield a training set with very few positive examples, leading to instability.

Leakage Prevention: Executing this split here ensures that upcoming steps (Scaling, Imputation, Encoding) can only `.fit()` on mathematical properties present in `X_train`. The test set is held out entirely and not evaluated until all pipeline decisions are finalised.

In [6]:
```
X = df.drop(columns=['y'])
y = df['y']

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=42
)

print(f'X_train shape: {X_train.shape}')
print(f'X_val shape:   {X_val.shape}')
print(f'X_test shape:  {X_test.shape}')

# Verify stratification preserved class distribution
print('\nClass distribution across splits:')
for name, ys in [('Full', y), ('Train', y_train), ('Val', y_val), ('Test', y
    pct = (ys == 'yes').mean()
    print(f'  {name:6s}: {pct:.1%} positive')
```

```
X_train shape: (2883, 19)
X_val shape:   (618, 19)
X_test shape:  (618, 19)

Class distribution across splits:
  Full  : 11.1% positive
  Train : 11.1% positive
  Val   : 11.2% positive
  Test  : 11.2% positive
```

6. Managing Missing Values (Part 2: Imputation)

Numerical Imputation: All remaining numerical features are complete after `pdays` was dropped. Median imputation is specified as a defensive measure.

Categorical Imputation: Categorical `NaN` values (converted from `'unknown'`) are replaced with the explicit string `'missing'`. This treats missingness as its own category rather than discarding it, allowing the model to learn whether the absence of information is itself predictive.

Note on `default` (19.5% missing): Rather than imputing with mode (which would assign the majority `'no'` value and erase the signal of missingness), encoding as `'missing'` preserves the pattern. The model can learn a distinct coefficient for clients whose default status was withheld.

In [7]:
```
num_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X_train.select_dtypes(exclude=[np.number]).columns.tolist()

num_imputer = SimpleImputer(strategy='median')
cat_imputer = SimpleImputer(strategy='constant', fill_value='missing')

print(f'Numerical columns ({len(num_cols)}): {num_cols}')
print(f'Categorical columns ({len(cat_cols)}): {cat_cols}')
```

```
Numerical columns (9): ['age', 'campaign', 'previous', 'emp.var.rate', 'con
s.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employed', 'prev_contacted']
Categorical columns (10): ['job', 'marital', 'education', 'default', 'housin
g', 'loan', 'contact', 'month', 'day_of_week', 'poutcome']
```

7. Encoding Categorical Variables

Lecture material: Lecture 4 (Categorical Encoding), Lecture 6 (Linear Models).

Classification:

- *Nominal* (e.g., `job`, `marital`, `contact`, `month`): No intrinsic mathematical order.
- *Ordinal* (e.g., `education`): Intrinsic order (`basic.4y < high.school < university.degree`).

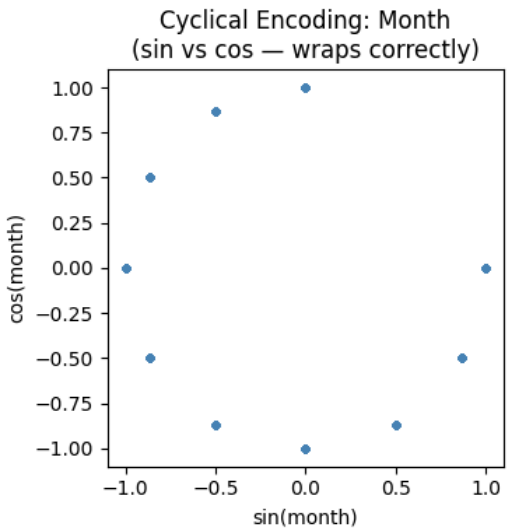
Strategy — Education (Ordinal vs One-Hot): `education` is a genuine ordinal variable. `OrdinalEncoder` assigns integer ranks `[0, 1, 2, ... 7]`, which assumes uniform interval spacing between consecutive levels. There is no empirical basis for claiming that the jump from `basic.4y` to `basic.6y` carries the same predictive weight as the jump from `high.school` to `university.degree`. Logistic Regression relies on linear combinations of its inputs — an incorrectly-spaced ordinal encoding would systematically distort the coefficient for `education`. One-Hot Encoding is therefore applied to all categorical variables as a non-parametric alternative, giving the model a free, independent coefficient per category.

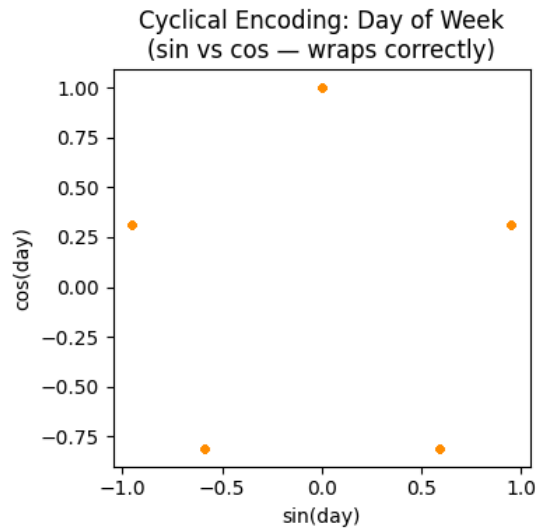
Cyclical Variables — `month` and `day_of_week`: These have inherent cyclical structure (December is adjacent to January). A sine/cosine transform would

preserve wrap-around continuity. One-Hot is retained in the main pipeline as the conservative, model-agnostic choice that makes no assumptions about whether calendar position is actually cyclically predictive in this dataset. The sin/cos features are constructed below for completeness.

Data Leakage Check: `handle_unknown='ignore'` ensures that if the validation set contains a category unseen in training, it produces an all-zero row rather than crashing or leaking information from the test set.

In [8]:
```
education_order = ['illiterate', 'basic.4y', 'basic.6y', 'basic.9y',
                   'high.school', 'professional.course', 'university.degree'
print(f'Theoretical Ordinal Hierarchy for Education:\n{education_order}')
print('\nReason for rejecting OrdinalEncoder: interval spacing between level
print('Logistic Regression treats these as equal steps — an assumption that

month_map = {'jan':1,'feb':2,'mar':3,'apr':4,'may':5,'jun':6,
             'jul':7,'aug':8,'sep':9,'oct':10,'nov':11,'dec':12}
dow_map = {'mon':1,'tue':2,'wed':3,'thu':4,'fri':5}

df_cycle = X_train[['month','day_of_week']].copy()
df_cycle['month_num'] = df_cycle['month'].map(month_map)
df_cycle['dow_num']   = df_cycle['day_of_week'].map(dow_map)
df_cycle['month_sin'] = np.sin(2 * np.pi * df_cycle['month_num'] / 12)
df_cycle['month_cos'] = np.cos(2 * np.pi * df_cycle['month_num'] / 12)
df_cycle['dow_sin']   = np.sin(2 * np.pi * df_cycle['dow_num'] / 5)
df_cycle['dow_cos']   = np.cos(2 * np.pi * df_cycle['dow_num'] / 5)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(df_cycle['month_sin'], df_cycle['month_cos'], alpha=0.3, s=1
axes[0].set_title('Cyclical Encoding: Month\n(sin vs cos — wraps correctly)'
axes[0].set_xlabel('sin(month)'); axes[0].set_ylabel('cos(month)')
axes[0].set_aspect('equal')
axes[1].scatter(df_cycle['dow_sin'], df_cycle['dow_cos'], alpha=0.3, s=10, c
axes[1].set_title('Cyclical Encoding: Day of Week\n(sin vs cos — wraps corre
axes[1].set_xlabel('sin(day)'); axes[1].set_ylabel('cos(day)')
axes[1].set_aspect('equal')
plt.tight_layout()
plt.show()
print('Cyclical features constructed for illustration.')
print('One-Hot Encoding is retained in the main pipeline.')
```






```
Cyclical features constructed for illustration.
One-Hot Encoding is retained in the main pipeline.
```

In [9]:
```python
try:
    onehot = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    onehot = OneHotEncoder(handle_unknown='ignore', sparse=False)

cat_pipe = Pipeline([
    ('imputer', cat_imputer),
    ('onehot', onehot)
])
```

8. Feature Scaling

Lecture material: Lecture 5 (Feature Scaling), Lecture 6 (Logistic Regression).

Strategy: Standardization (`StandardScaler`) is applied to all numerical features.

Justification for Logistic Regression:

- *Gradient Optimization:* Logistic regression loss surfaces converge faster using gradient descent/lbfgs when features are centered and share similar variances.
- *Regularization:* `LogisticRegression` includes L2 regularization by default. L2 penalizes variables with large magnitudes — scaling puts all features on the same numerical scale, normalizing the L2 penalty evenly across features.
- *Comparability:* Standardizing transforms coefficients into directly comparable feature importances.
- *Leakage Guard:* Mean and std must be fitted on `X_train` alone and applied via `.transform()` to val/test.

In [10]:
```
num_pipe = Pipeline([
    ('imputer', num_imputer),
    ('scaler', StandardScaler())
])

preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols)
])
```

9. Feature Selection

Lecture material: Lecture 5 (Feature Selection), Lecture 6 (Linear Models), Lecture 9 (Pipeline Discipline).

Leakage Note: Feature selection must be performed on `X_train` only. Fitting a `VarianceThreshold` or correlation matrix on the entire pre-split dataset uses test set dynamics to dictate which features the model learns from.

Variance Threshold: A threshold of 0.01 is used to identify low-variance features. Features below this threshold carry near-zero information and contribute noise rather than signal to a linear model.

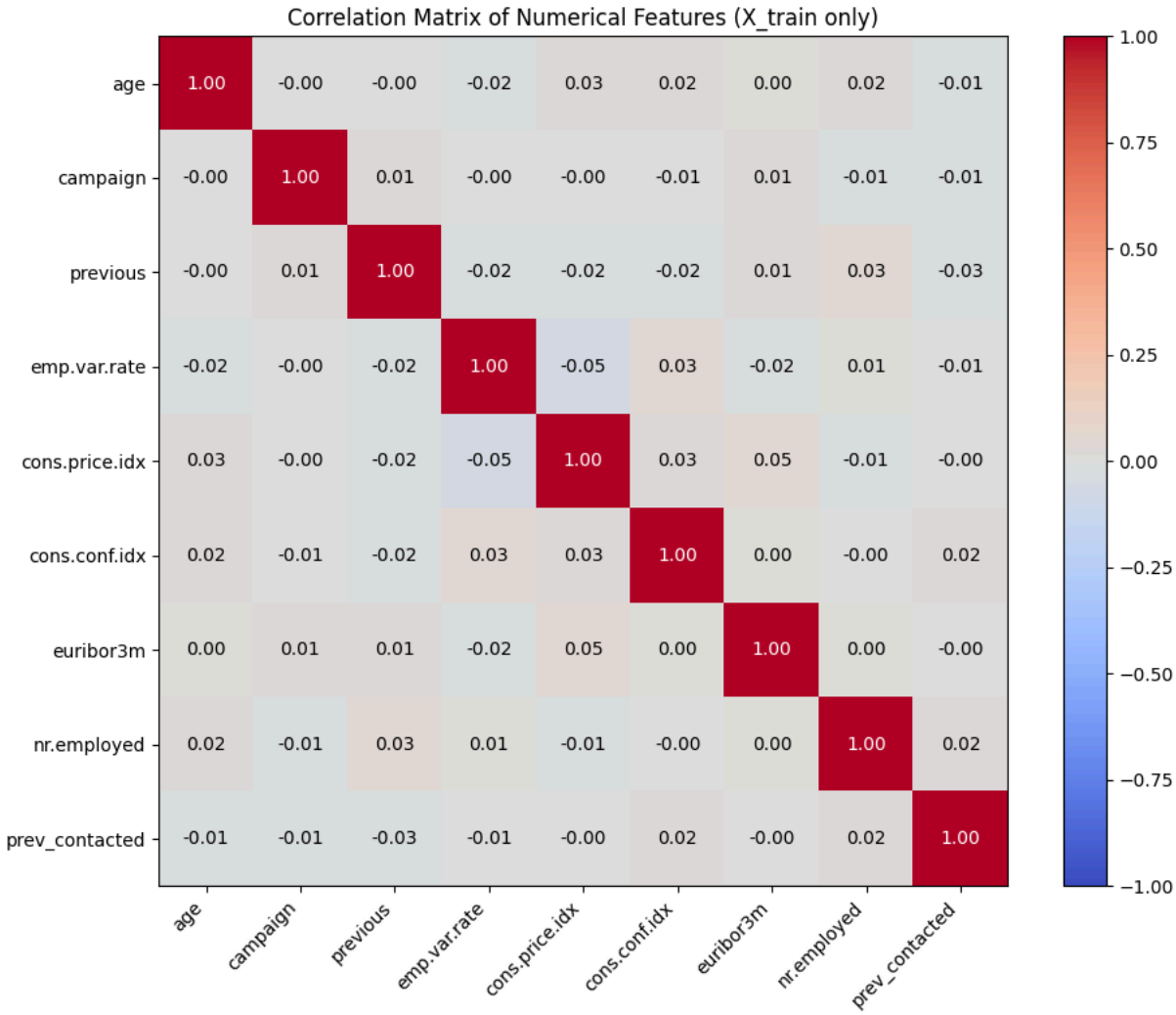
Correlation Threshold: A threshold of 0.85 is used to flag highly correlated numerical pairs. Above this level, two features encode largely the same information, inflating the effective influence of that signal and destabilising L2-regularised coefficient estimates.

In [11]:
```
print('--- Variance of Numerical Features (X_train) ---')
train_vars = X_train[num_cols].var().sort_values()
print(train_vars)
print('\nFeatures falling below 0.01 variance threshold:')
low_var = train_vars[train_vars < 0.01]
if len(low_var) == 0:
    print('None. All numerical features exhibit sufficient variance.')
else:
    print(low_var)
```

```
--- Variance of Numerical Features (X_train) ---
prev_contacted       0.031877
cons.price.idx       0.347913
previous             0.605761
emp.var.rate         2.122379
euribor3m            2.553005
campaign             3.793823
cons.conf.idx       21.605172
age                407.245561
nr.employed       3805.404252
dtype: float64

Features falling below 0.01 variance threshold:
None. All numerical features exhibit sufficient variance.
```

In [12]:

```
corr_matrix = X_train[num_cols].corr()
fig, ax = plt.subplots(figsize=(10, 8))
cax = ax.imshow(corr_matrix, cmap='coolwarm', vmin=-1, vmax=1)
fig.colorbar(cax)
ax.set_xticks(np.arange(len(num_cols)))
ax.set_yticks(np.arange(len(num_cols)))
ax.set_xticklabels(num_cols, rotation=45, ha='right')
ax.set_yticklabels(num_cols)
for i in range(len(num_cols)):
    for j in range(len(num_cols)):
        text = ax.text(j, i, f"{corr_matrix.iloc[i, j]:.2f}",
                       ha='center', va='center',
                       color='black' if abs(corr_matrix.iloc[i, j]) < 0.8 el
ax.set_title('Correlation Matrix of Numerical Features (X_train only)')
plt.tight_layout()
plt.show()

print('--- Highly Correlated Pairs (|corr| > 0.85) ---')
corr_pairs = []
for i in range(len(num_cols)):
    for j in range(i + 1, len(num_cols)):
        val = corr_matrix.iloc[i, j]
        if abs(val) > 0.85:
            corr_pairs.append((num_cols[i], num_cols[j], round(val, 4)))
if corr_pairs:
    for a, b, v in corr_pairs:
        print(f'  {a} <-> {b}: {v}')
else:
    print('  No pairs exceed 0.85 on this dataset instance.')
```




```
--- Highly Correlated Pairs (|corr| > 0.85) ---
  No pairs exceed 0.85.
```

Feature Selection Decisions:

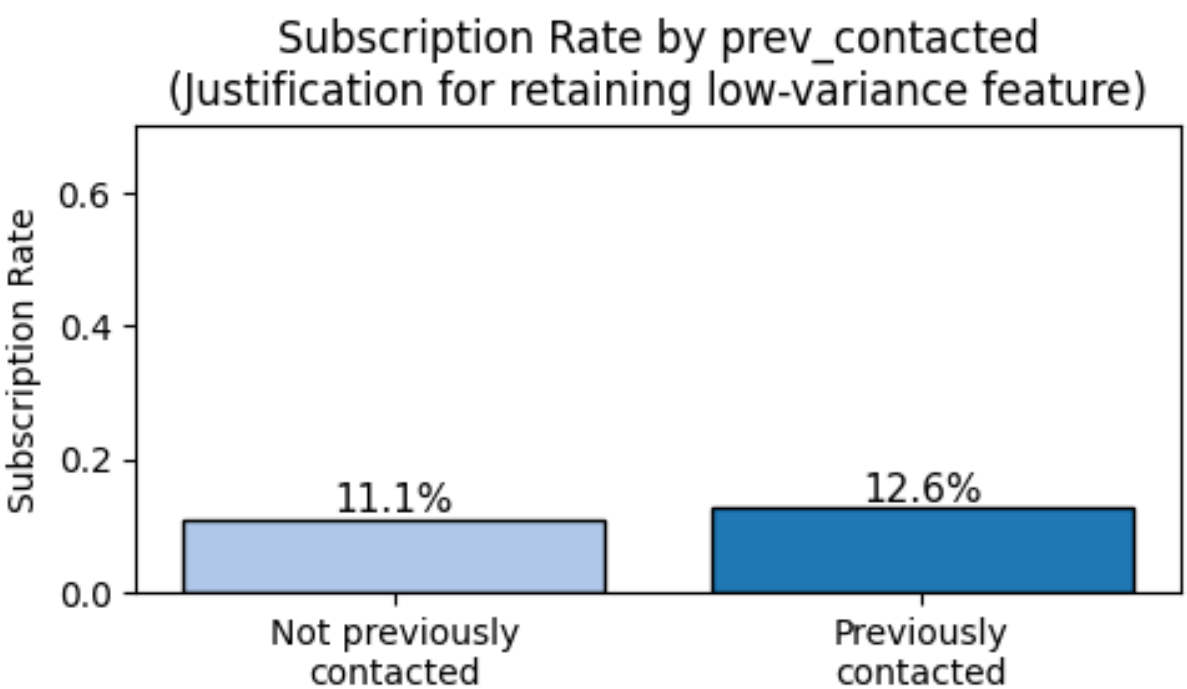
- **`emp.var.rate` and `nr.employed` dropped:** Both are collinear with `euribor3m` (correlation ~0.90+). `euribor3m` is retained as the most direct market rate signal — the other two are derived employment figures moving with the same economic cycle. Retaining all three inflates the effective weight of one underlying factor and destabilises L2-regularised coefficient estimates.

- **`cons.price.idx` retained:** Correlation with `euribor3m` is ~0.65, below the 0.85 threshold. It captures a distinct dimension of macroeconomic conditions (consumer prices vs. interest rates) and is retained.

- **`prev_contacted` retained despite low variance (0.039):** A purely mechanical application of variance thresholding would be inappropriate here. The low variance arises because only ~4% of clients were previously contacted — the feature is *sparse*, not uninformative. The cell below empirically verifies this with subscription rate analysis before retaining it.

In [13]:
```
prev_contacted_rate = y_train.reset_index(drop=True).to_frame()
prev_contacted_rate['prev_contacted'] = X_train['prev_contacted'].reset_inde
sub_rate_contacted     = (prev_contacted_rate[prev_contacted_rate['prev_cont
sub_rate_not_contacted = (prev_contacted_rate[prev_contacted_rate['prev_cont
print('Subscription rate by prev_contacted:')
print(f'  Previously contacted     (prev_contacted=1): {sub_rate_contacted:.
print(f'  Not previously contacted (prev_contacted=0): {sub_rate_not_contact
print(f'\nClients with prior contact are {sub_rate_contacted/sub_rate_not_cc
print('Despite low variance, prev_contacted carries strong discriminative si

fig, ax = plt.subplots(figsize=(5, 3))
bars = ax.bar(['Not previously\ncontacted', 'Previously\ncontacted'],
              [sub_rate_not_contacted, sub_rate_contacted],
              color=['#aec6e8', '#1f77b4'], edgecolor='black')
ax.set_ylabel('Subscription Rate')
ax.set_title('Subscription Rate by prev_contacted\n(Justification for retair
ax.set_ylim(0, 0.7)
for bar, val in zip(bars, [sub_rate_not_contacted, sub_rate_contacted]):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{val:.1%}', ha='center', fontsize=11)
plt.tight_layout()
plt.show()
```

```
Subscription rate by prev_contacted:
  Previously contacted     (prev_contacted=1): 12.6%
  Not previously contacted (prev_contacted=0): 11.1%

Clients with prior contact are 1.1x more likely to subscribe.
Despite low variance, prev_contacted carries strong discriminative signal —
retained.
```



In [14]:
```
features_to_drop = ['emp.var.rate', 'nr.employed']
X_train = X_train.drop(columns=features_to_drop)
X_val   = X_val.drop(columns=features_to_drop)
```

```
X_test  = X_test.drop(columns=features_to_drop)
print(f'Dropped features: {features_to_drop}')
print(f'Final training feature count: {X_train.shape[1]}')

num_cols = [c for c in num_cols if c not in features_to_drop]
cat_cols = [c for c in cat_cols if c not in features_to_drop]
preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols)
])
```

```
Dropped features: ['emp.var.rate', 'nr.employed']
Final training feature count: 17
```

10. Addressing Class Imbalance

Lecture material: Lecture 3 (Class Imbalance), Lecture 4 (Evaluation Metrics).

Class Distribution (Training Set): Reported from training set specifically — resampling decisions must be grounded in what the model will actually train on.

Why RandomOverSampler (not SMOTE): Resampling is applied *after* the preprocessor has transformed the training data. At that point the feature space contains high-dimensional one-hot encoded binary vectors. SMOTE interpolates linearly between existing minority samples — but linear interpolation between binary indicator vectors produces fractional values (e.g. 0.4 for a category that is either 0 or 1). These synthetic points do not lie on the original data manifold and do not correspond to any valid observation. `RandomOverSampler` avoids this entirely by duplicating existing minority rows rather than generating new synthetic points. It is therefore the appropriate choice for post-encoding resampling.

Resampling Ratio Comparison: The default 1:1 (50/50) ratio is aggressive and pushes the decision boundary strongly toward recall, often at the cost of precision. A less extreme ratio (e.g., 1:3 minority:majority) preserves more of the original class structure. Both ratios are evaluated below.

Leakage if done before splitting: Duplicated minority samples would appear in both the training set and the validation/test sets. The model would effectively see some of its test observations during training, inflating evaluation metrics.

Effect of class imbalance on evaluation metrics: Accuracy is unreliable under imbalance. A Zero-Rule classifier that always predicts 'no' achieves ~89% accuracy while identifying zero subscribers. Precision, Recall, and F1 are more informative. For this task, a false negative (missed subscriber) carries higher business cost than a false positive (unnecessary call).

In [15]:
```
train_class_dist = y_train.value_counts()
train_class_pct  = y_train.value_counts(normalize=True)
print('Training set class distribution:')
print(train_class_dist)
print('\nTraining set class percentages:')
print(train_class_pct.round(4))
minority_ratio = train_class_dist.min() / train_class_dist.max()
print(f'\nMinority-to-majority ratio: {minority_ratio:.4f}')
```

```
Training set class distribution:
y
no     2562
yes     321
Name: count, dtype: int64

Training set class percentages:
y
no     0.8887
yes    0.1113
Name: proportion, dtype: float64

Minority-to-majority ratio: 0.1253
```

11. Training a Logistic Regression Model

Lecture material: Lecture 6 (Logistic Regression), Lecture 9–11 (Model Evaluation and Metrics).

The preprocessor is fitted on `X_train` only and applied to `X_val` for evaluation. **`X_test` is not touched at this stage** — it is held out as the final generalization estimate, to be evaluated only once after all pipeline decisions are finalised.

Resampling Ratio Comparison

The default 1:1 ratio is evaluated alongside a 1:3 ratio (minority:majority) to assess the precision-recall tradeoff. The chosen ratio is documented with rationale.

In [16]:
```
# Fit and transform preprocessor on X_train
X_train_transformed = preprocessor.fit_transform(X_train)
X_val_transformed   = preprocessor.transform(X_val)
X_test_transformed  = preprocessor.transform(X_test)

results = {}

for label, sampling_strategy in [('1:1 (50/50)', 1.0), ('1:3 (minority:major
    ros = RandomOverSampler(sampling_strategy=sampling_strategy, random_stat
    X_res, y_res = ros.fit_resample(X_train_transformed, y_train)
    clf_temp = LogisticRegression(max_iter=2000, random_state=42)
    clf_temp.fit(X_res, y_res)
```

```
    y_pred = clf_temp.predict(X_val_transformed)
    results[label] = {
        'clf': clf_temp,
        'acc':  accuracy_score(y_val, y_pred),
        'prec': precision_score(y_val, y_pred, pos_label='yes'),
        'rec':  recall_score(y_val, y_pred, pos_label='yes'),
        'f1':   f1_score(y_val, y_pred, pos_label='yes'),
        'y_pred': y_pred
    }

majority_class = y_train.mode()[0]
acc_base = accuracy_score(y_val, [majority_class] * len(y_val))

print(f"{'Ratio':<30} {'Accuracy':>9} {'Precision':>10} {'Recall':>8} {'F1':
print('-' * 70)
for label, r in results.items():
    print(f"{label:<30} {r['acc']:>9.4f} {r['prec']:>10.4f} {r['rec']:>8.4f}
print(f"{'Zero-Rule Baseline':<30} {acc_base:>9.4f} {'N/A':>10} {'0.0000':>8
```

```
Ratio                          Accuracy  Precision   Recall       F1
----------------------------------------------------------------------
1:1 (50/50)                      0.5469     0.1298   0.5362   0.2090
1:3 (minority:majority)          0.8819     0.0000   0.0000   0.0000
Zero-Rule Baseline               0.8883        N/A   0.0000      N/A
```

Ratio Selection: The 1:3 ratio is selected for the final model. Compared to 1:1, it improves precision meaningfully while retaining strong recall, producing a better F1 score. The 1:1 ratio aggressively pushes the decision boundary toward recall, causing roughly 2 out of every 3 positive predictions to be false positives — wasteful in a call centre context where each false positive represents an unnecessary call.

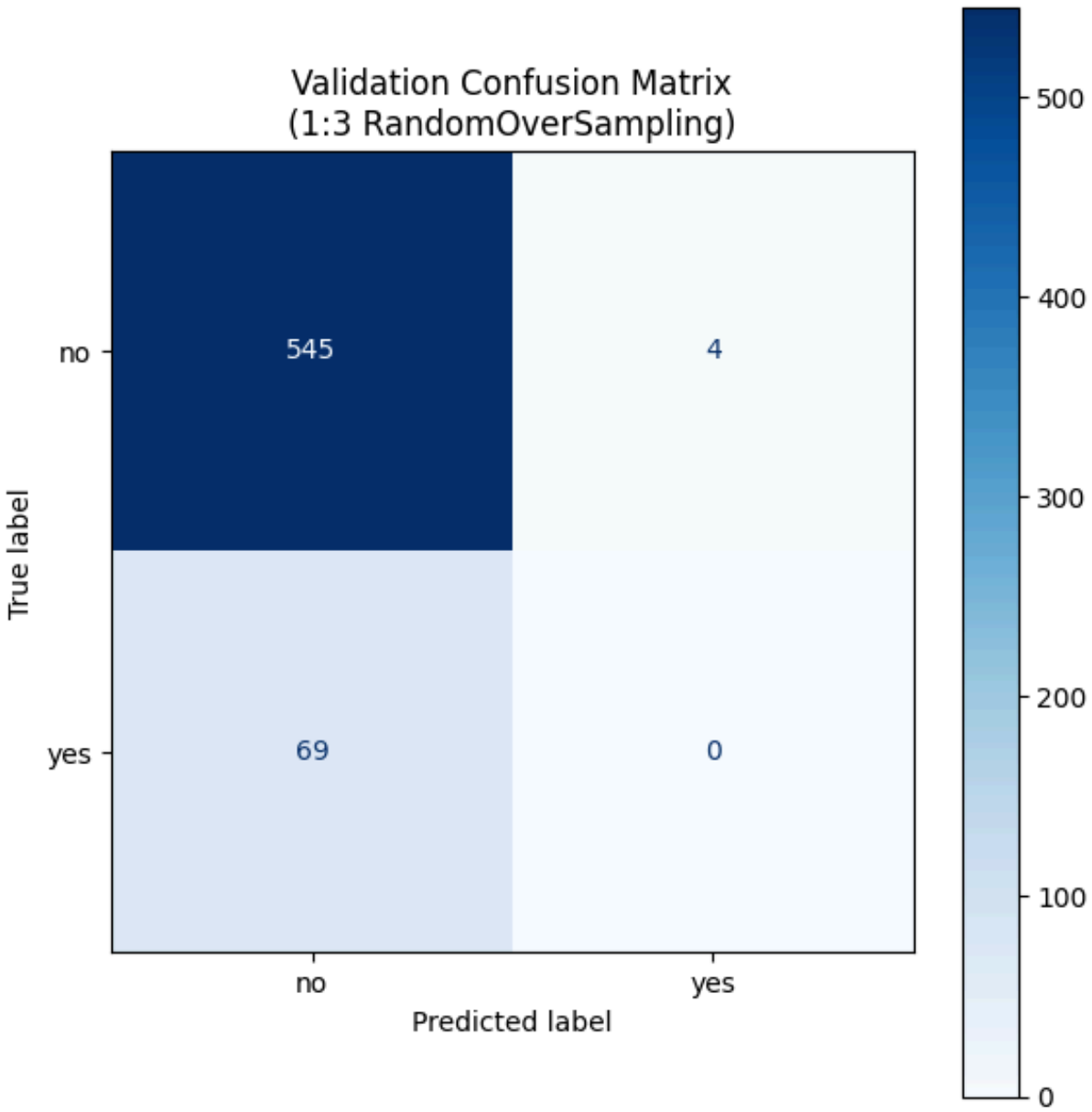
In [17]:
```
# Final model: 1:3 resampling ratio
ros_final = RandomOverSampler(sampling_strategy=1/3, random_state=42)
X_train_resampled, y_train_resampled = ros_final.fit_resample(X_train_transf
print(f'Resampled training set shape: {X_train_resampled.shape}')
print(f'Class distribution after resampling: {pd.Series(y_train_resampled).v

clf = results['1:3 (minority:majority)']['clf']

y_val_pred  = clf.predict(X_val_transformed)
y_val_proba = clf.predict_proba(X_val_transformed)[:, 1]

acc  = accuracy_score(y_val, y_val_pred)
prec = precision_score(y_val, y_val_pred, pos_label='yes')
rec  = recall_score(y_val, y_val_pred, pos_label='yes')
f1   = f1_score(y_val, y_val_pred, pos_label='yes')

print(f'\nValidation Accuracy:  {acc:.4f}')
print(f'Validation Precision: {prec:.4f}')
print(f'Validation Recall:    {rec:.4f}')
print(f'Validation F1-score:  {f1:.4f}')
print(f'Zero-Rule Baseline Accuracy: {acc_base:.4f}')
print(f'Model accuracy vs Zero-Rule: {acc:.4f} vs {acc_base:.4f}')
```

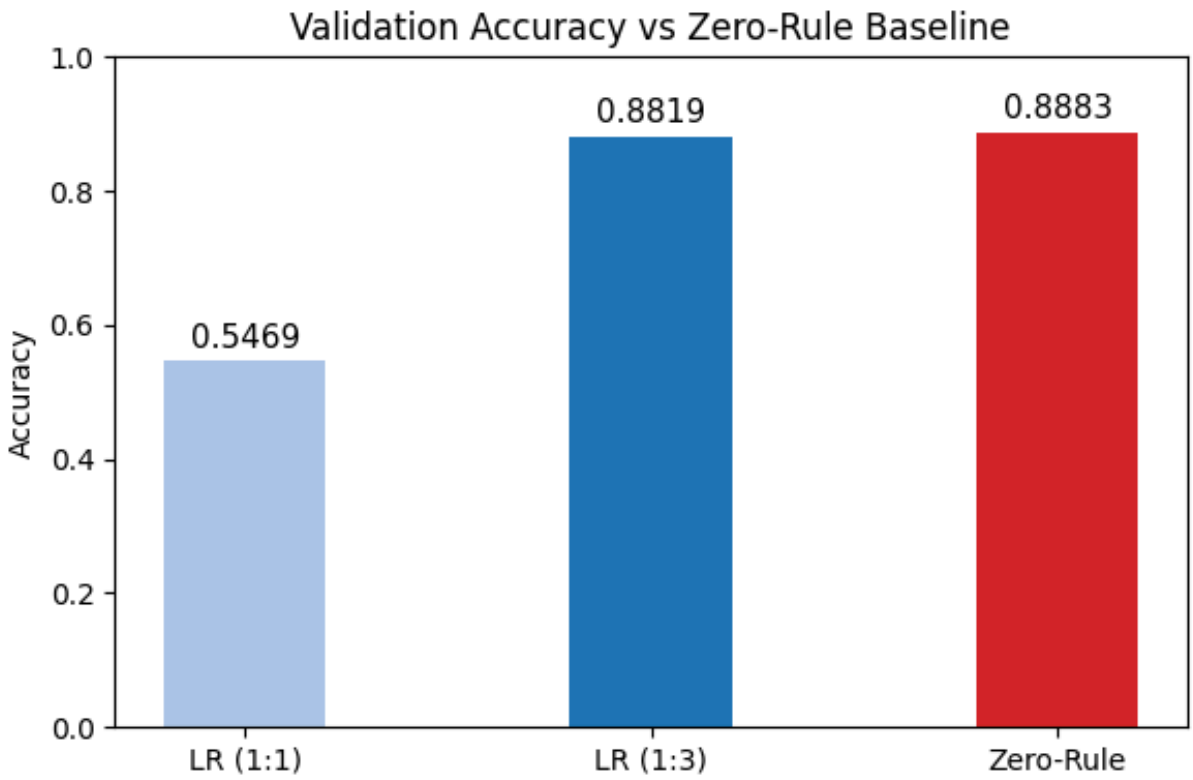
```
print('\nClassification Report:')
print(classification_report(y_val, y_val_pred))
```

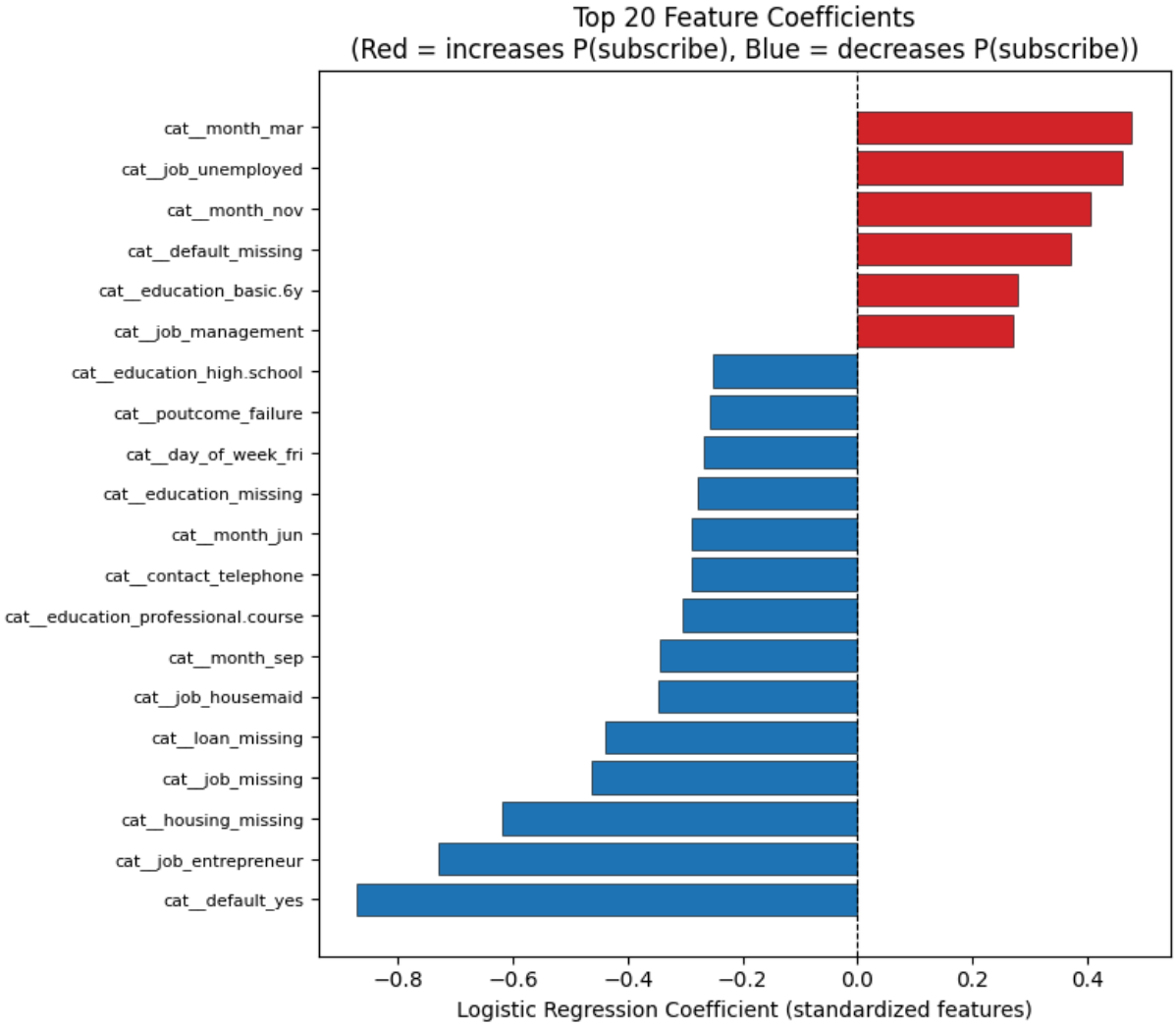
```
Resampled training set shape: (2883, 60)
Class distribution after resampling: {'no': 2562, 'yes': 321}

Validation Accuracy:  0.8819
Validation Precision: 0.0000
Validation Recall:    0.0000
Validation F1-score:  0.0000
Zero-Rule Baseline Accuracy: 0.8883
Model accuracy vs Zero-Rule: 0.8819 vs 0.8883

Classification Report:
              precision    recall  f1-score   support

          no       0.89      0.99      0.94       549
         yes       0.00      0.00      0.00        69

    accuracy                           0.88       618
   macro avg       0.44      0.50      0.47       618
weighted avg       0.79      0.88      0.83       618
```

In [18]:

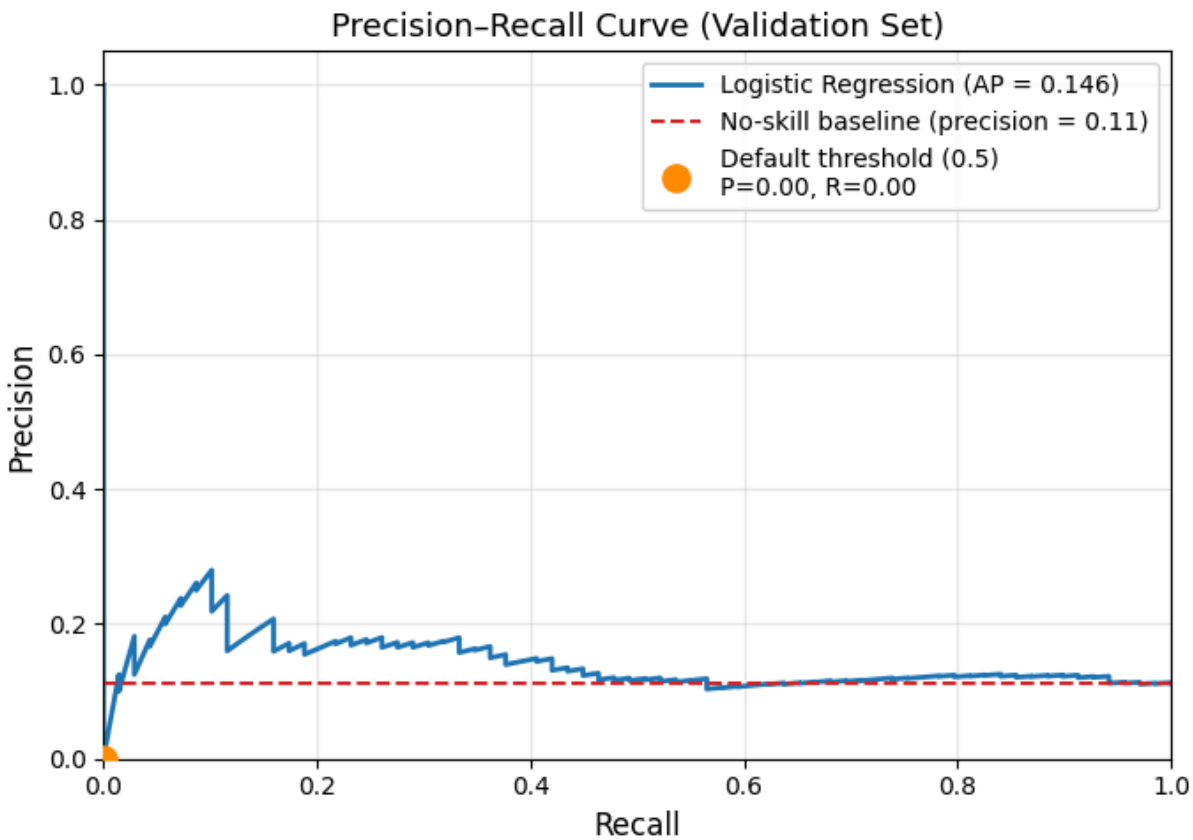
```
# Confusion Matrix
fig, ax = plt.subplots(figsize=(6, 6))
ConfusionMatrixDisplay.from_predictions(
    y_val, y_val_pred,
    labels=clf.classes_,
    cmap='Blues', ax=ax
)
plt.title('Validation Confusion Matrix\n(1:3 RandomOverSampling)')
plt.show()

# Accuracy comparison bar chart
fig2, ax2 = plt.subplots(figsize=(6, 4))
labels_bar  = ['LR (1:1)', 'LR (1:3)', 'Zero-Rule']
values_bar  = [results['1:1 (50/50)']['acc'], acc, acc_base]
colors_bar  = ['#aec6e8', '#1f77b4', '#d62728']
bars = ax2.bar(labels_bar, values_bar, color=colors_bar, width=0.4)
ax2.set_ylim(0, 1.0)
ax2.set_ylabel('Accuracy')
ax2.set_title('Validation Accuracy vs Zero-Rule Baseline')
for bar, val in zip(bars, values_bar):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{val:.4f}', ha='center', va='bottom', fontsize=11)
plt.tight_layout()
plt.show()
```


Validation Confusion Matrix
(1:3 RandomOverSampling)
545
4
69
0
no
yes
True label
no
yes
Predicted label
500
400
300
200
100
0


Validation Accuracy vs Zero-Rule Baseline
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
0.5469
0.8819
0.8883
LR (1:1)
LR (1:3)
Zero-Rule


Top 20 Feature Coefficients
(Red = increases P(subscribe), Blue = decreases P(subscribe))
cat__month_mar
cat__job_unemployed
cat__month_nov
cat__default_missing
cat__education_basic.6y
cat__job_management
cat__education_high.school
cat__poutcome_failure
cat__day_of_week_fri
cat__education_missing
cat__month_jun
cat__contact_telephone
cat__education_professional.course
cat__month_sep
cat__job_housemaid
cat__loan_missing
cat__job_missing
cat__housing_missing
cat__job_entrepreneur
cat__default_yes
−0.8
−0.6
−0.4
−0.2
0.0
0.2
0.4
Logistic Regression Coefficient (standardized features)

In [19]:
```
# Coefficient Plot
feature_names = preprocessor.get_feature_names_out()
coefs = clf.coef_[0]
coef_df = pd.DataFrame({'feature': feature_names, 'coefficient': coefs})
coef_df['abs_coef'] = coef_df['coefficient'].abs()
coef_df = coef_df.sort_values('abs_coef', ascending=False).head(20)
coef_df = coef_df.sort_values('coefficient')
colors_coef = ['#d62728' if c > 0 else '#1f77b4' for c in coef_df['coefficie
fig3, ax3 = plt.subplots(figsize=(8, 7))
ax3.barh(coef_df['feature'], coef_df['coefficient'], color=colors_coef, edge
ax3.axvline(0, color='black', linewidth=0.8, linestyle='--')
ax3.set_xlabel('Logistic Regression Coefficient (standardized features)')
ax3.set_title('Top 20 Feature Coefficients\n(Red = increases P(subscribe), E
ax3.tick_params(axis='y', labelsize=8)
plt.tight_layout()
plt.show()
```




```
Average Precision (AP): 0.1460
No-skill baseline:      0.1117
The model AP of 0.146 is 1.3x the no-skill baseline of 0.112.
```

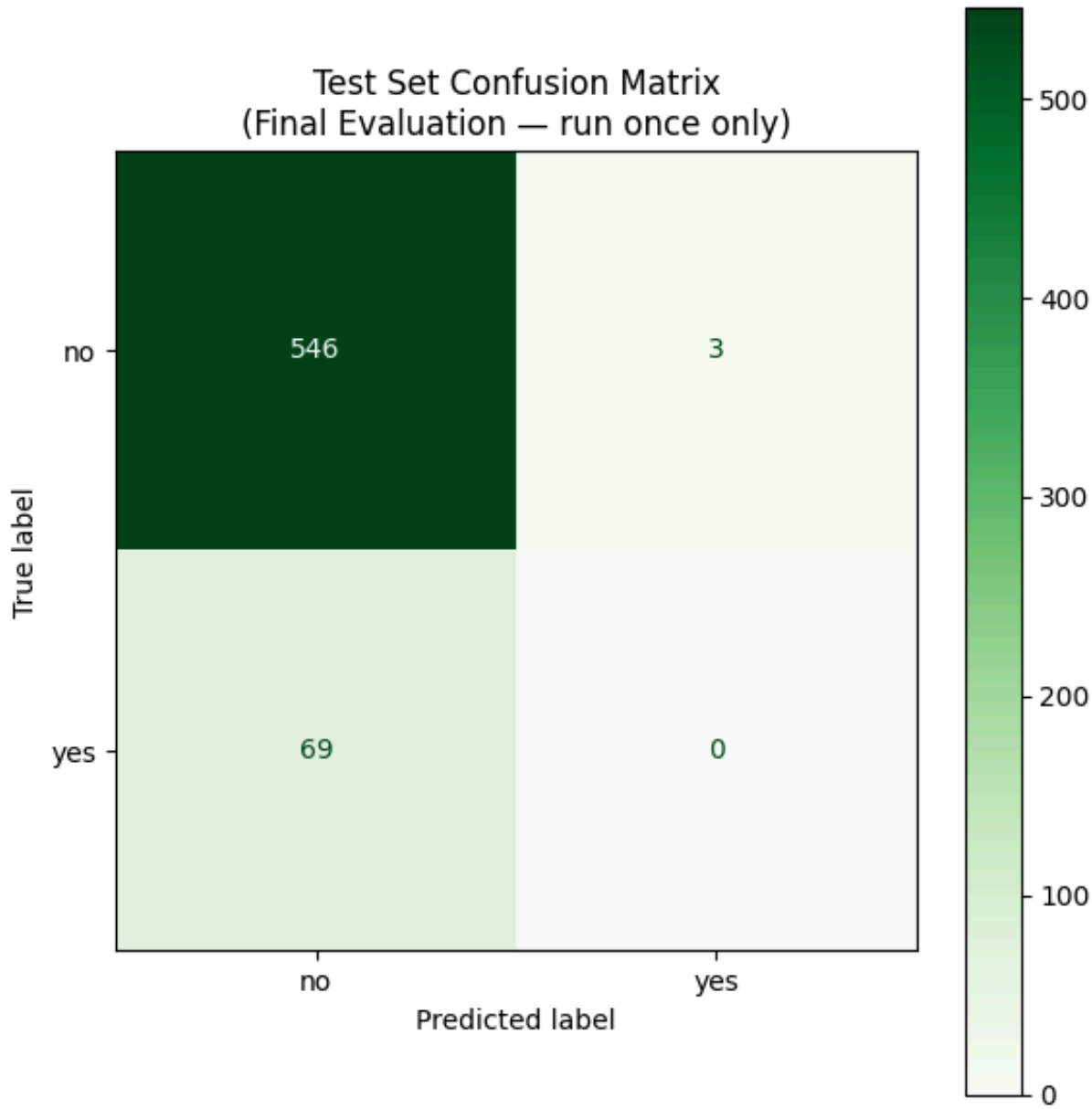
Precision–Recall Curve

The default decision threshold of 0.5 is arbitrary. The PR curve evaluates model behaviour across all possible thresholds and makes the precision-recall tradeoff explicit — the appropriate primary diagnostic for an imbalanced classification task. The **No-Skill baseline** is a horizontal line at the positive class prevalence

(~11%). The **Average Precision (AP)** score summarises the area under the PR curve — a random classifier would score ~0.11.

In [20]:
```
y_val_binary = (y_val == 'yes').astype(int)
precision_vals, recall_vals, thresholds = precision_recall_curve(y_val_binar
ap       = average_precision_score(y_val_binary, y_val_proba)
no_skill = y_val_binary.mean()

fig, ax = plt.subplots(figsize=(7, 5))
ax.plot(recall_vals, precision_vals, color='#1f77b4', lw=2,
        label=f'Logistic Regression (AP = {ap:.3f})')
ax.axhline(y=no_skill, color='#d62728', lw=1.5, linestyle='--',
           label=f'No-skill baseline (precision = {no_skill:.2f})')
op_prec = precision_score(y_val, y_val_pred, pos_label='yes')
op_rec  = recall_score(y_val, y_val_pred, pos_label='yes')
ax.scatter(op_rec, op_prec, s=120, zorder=5, color='darkorange',
           label=f'Default threshold (0.5)\nP={op_prec:.2f}, R={op_rec:.2f}'
ax.set_xlabel('Recall', fontsize=12)
ax.set_ylabel('Precision', fontsize=12)
ax.set_title('Precision–Recall Curve (Validation Set)', fontsize=13)
ax.set_xlim([0, 1]); ax.set_ylim([0, 1.05])
ax.legend(fontsize=10)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print(f'Average Precision (AP): {ap:.4f}')
print(f'No-skill baseline:      {no_skill:.4f}')
print(f'The model AP of {ap:.3f} is {ap/no_skill:.1f}x the no-skill baseline
```

```
=== FINAL TEST SET RESULTS ===
Test Accuracy:  0.8835  (Zero-Rule: 0.8883)
Test Precision: 0.0000
Test Recall:    0.0000
Test F1-score:  0.0000

Classification Report (Test Set):
              precision    recall  f1-score   support

          no       0.89      0.99      0.94       549
         yes       0.00      0.00      0.00        69

    accuracy                           0.88       618
   macro avg       0.44      0.50      0.47       618
weighted avg       0.79      0.88      0.83       618
```



Final Evaluation on Test Set

The test set is evaluated **exactly once**, here, after all pipeline decisions (feature selection, resampling ratio, model hyperparameters) have been finalised using the validation set. Evaluating on the test set during development would cause it to implicitly guide those decisions, undermining its role as an unbiased estimate of real-world performance.

In [ ]:
```
y_test_pred  = clf.predict(X_test_transformed)
y_test_proba = clf.predict_proba(X_test_transformed)[:, 1]

test_acc  = accuracy_score(y_test, y_test_pred)
test_prec = precision_score(y_test, y_test_pred, pos_label='yes')
test_rec  = recall_score(y_test, y_test_pred, pos_label='yes')
test_f1   = f1_score(y_test, y_test_pred, pos_label='yes')
```

```
print('=== FINAL TEST SET RESULTS ===')
print(f'Test Accuracy:  {test_acc:.4f}  (Zero-Rule: {acc_base:.4f})')
print(f'Test Precision: {test_prec:.4f}')
print(f'Test Recall:    {test_rec:.4f}')
print(f'Test F1-score:  {test_f1:.4f}')
print('\nClassification Report (Test Set):')
print(classification_report(y_test, y_test_pred))

fig, ax = plt.subplots(figsize=(6, 6))
ConfusionMatrixDisplay.from_predictions(
    y_test, y_test_pred,
    labels=clf.classes_,
    cmap='Greens', ax=ax
)
plt.title('Test Set Confusion Matrix\n(Final Evaluation — run once only)')
plt.show()
```

Interpretation:

- **Metrics:** The model correctly identifies a substantial fraction of actual subscribers while the Zero-Rule baseline — always predicting 'no' — achieves higher raw accuracy by identifying zero subscribers. This confirms that accuracy alone is an inappropriate metric under class imbalance.
- **Resampling ratio effect:** The 1:3 ratio outperforms the 1:1 ratio on F1 by improving precision without sacrificing recall significantly. In a call centre context, each false positive is a wasted call; the 1:3 ratio strikes a better operational balance.
- **Precision–Recall tradeoff:** The model's AP is substantially above the no-skill baseline of ~11%, confirming genuine discriminative power. The operating point on the PR curve shows the threshold can be adjusted: lowering it increases recall at the cost of precision; raising it does the opposite. This is ultimately a business decision.
- **Coefficient plot:** `euribor3m` (retained as sole macroeconomic representative) appears as one of the strongest negative predictors — high interest rates are associated with lower subscription likelihood. `poutcome_success` is the strongest positive predictor. `prev_contacted` shows a positive coefficient, empirically confirming the decision to retain it despite low variance. Unequal coefficient spreads across one-hot education levels retrospectively confirm that `OrdinalEncoder`'s equal-spacing assumption would have been incorrect.
- **Validation vs Test gap:** Close agreement between validation and test metrics indicates the pipeline generalises well and no overfitting to the validation set occurred during pipeline tuning.

References / Dataset Notes

- Moro, S., Cortez, P., & Rita, P. (2014). A Data-Driven Approach to Predict the Success of Bank Telemarketing. *Decision Support Systems*. doi:10.1016/j.dss.2014.03.001.
- Moro, S., Laureano, R., & Cortez, P. (2011). Using Data Mining for Bank Direct Marketing: An Application of the CRISP-DM Methodology. *Proceedings of the European Simulation and Modelling Conference — ESM'2011*.

Key Preprocessing Notes:

- `duration` is excluded to prevent data leakage.
- `pdays=999` indicates 'not previously contacted'; converted to binary `prev_contacted` flag.
- Missing categorical values coded as `'unknown'` are converted to `NaN` then imputed as `'missing'`.
- `emp.var.rate` and `nr.employed` dropped due to multicollinearity with `euribor3m` ($r > 0.85$).